



DIGITAL IDENTITY VERIFICATION SYSTEM

By:



[DATE]

Words Glossary:

Credentials: A piece of any kind of physical documentations or digital evidence that used to testify and verify an individual identity, qualifications, or authority.

1 Introduction

1.1 Background

Digital identity systems are fundamental to modern digital infrastructure because they enable authentication, authorization, and accountability in online services. According to identity management research, secure verification is essential for preventing fraud and ensuring trust in distributed systems (NIST, 2017).

Existing credential verification systems are largely based on centralized architectures, where a single trusted authority is responsible for issuing, storing, and validating credentials. For example, national identity systems such as Aadhaar and European electronic identity (eID) platforms rely on centralized databases to manage citizen identity information. Similarly, university certificate verification portals and online transcript systems maintain centralized student credential records under institutional control. In the corporate sector, authentication services such as Google and Microsoft Single Sign-On (SSO) systems centralize user authentication and access management within a single organizational authority. Banking systems also employ centralized Know Your Customer (KYC) databases to store and verify customer identities. Although these systems are widely adopted due to their simplicity, scalability, and administrative control, they introduce limitations such as single points of failure, vulnerability to data breaches, service outages, and reduced user autonomy in controlling personal credentials (NIST, 2017; ISO/IEC 24760).

Recent research in digital identity management has increasingly emphasized the adoption of distributed architectures integrated with advanced cryptographic security mechanisms to overcome the limitations of centralized credential verification systems (NIST, 2017; Satybaldy et al., 2022). Unlike centralized models, distributed identity management approaches minimize dependency on a single trusted authority by distributing credential issuance and verification responsibilities across multiple entities within the network (ISO/IEC 24760; W3C DID Core Specification).

Real-world implementations of this approach can be observed in decentralized identity initiatives such as Hyperledger Indy (Hyperledger Foundation, 2020) and European Union digital identity initiatives under the European Blockchain Services Infrastructure (EBSI), which implements Self-Sovereign Identity principles (European Commission, 2020a). Similarly, the European Union's electronic identity framework under the eIDAS Regulation promotes interoperable and legally recognized digital identity verification across member states (European Commission, 2014/2020b).

These systems distribute trust and verification processes among multiple participants rather than relying entirely on a centralized database or authority. Consequently, this decentralized structure enhances system resilience, reduces single points of failure, improves transparency, and strengthens overall trust management in digital environments (Zyskind et al., 2015).

Against this background, the present study focuses on the design and implementation of a distributed digital credential verification system leveraging the Django framework in combination with RSA cryptographic techniques. The proposed system is based on a role-driven architecture that defines three primary entities: issuer, holder, and verifier. This model is designed to support secure and verifiable credential exchange within a distributed environment by ensuring proper separation of responsibilities among participating entities. By integrating cryptographic assurance with a structured distributed workflow, the system aims to enhance the integrity, authenticity, and security of credential verification processes while reducing reliance on centralized authorities.

1.2 Problem Statement

Despite significant advancements in digital identity systems, a substantial proportion of existing credential verification frameworks continue to rely on centralized architectures. These systems depend on a single trusted authority for issuing, storing, and validating credentials, which introduces critical security and trust-related vulnerabilities. Centralized models are inherently susceptible to unauthorized data access, credential manipulation, and service disruptions due to single points of failure (NIST, 2017).

Furthermore, such systems often provide limited transparency in the credential verification process and restrict user autonomy over the control, sharing, and lifecycle of digital credentials. In contrast, emerging standards such as the W3C Verifiable Credentials model illustrate a more decentralized approach, where credentials are issued by trusted entities but verified independently without requiring continuous reliance on a central authority. For example, in a W3C Verifiable Credentials system, a university can issue a digital degree credential, and the holder can present it to an employer, who independently verifies its authenticity through cryptographic proof without contacting the issuing institution directly (W3C, 2022).

These limitations of centralized systems highlight the need for alternative architectures that distribute trust while maintaining strong security guarantees. Therefore, this study addresses the problem of designing and implementing a secure, reliable, and distributed credential verification system using cryptographic techniques to ensure data integrity, authentication security, and controlled access to digital credentials within a decentralized framework (Satybaldy et al., 2022; W3C, 2022).

1.3 Research Questions

RQ1: How can a distributed role-based architecture combined with RSA cryptography improve the security and reliability of digital credential verification systems?

RQ2: What are the limitations of the proposed system in terms of scalability and decentralization?

1.4 Objectives

1.4.1 General Objectives

The general objective of this study is to design and evaluate a secure and efficient distributed digital credential verification system by integrating the Django framework with RSA cryptographic techniques to enhance the security, reliability, and trustworthiness of digital credential verification processes.

1.4.2 Specific Objectives

- To design and implement a role-based credential verification model consisting of three primary entities: issuer, holder, and verifier, based on distributed identity management principles.
- To apply RSA cryptographic techniques for secure credential signing, encryption, and authentication.
- To ensure the integrity, authenticity, and controlled access of digital credentials within the proposed system.
- To reduce dependency on centralized verification authorities by adopting a distributed system architecture aligned with modern decentralized identity approaches such as W3C Verifiable Credentials.
- To evaluate the performance, security strengths, and limitations of the proposed system in comparison to traditional centralized credential verification systems.

1.5 Scope

This study focuses on the design and implementation of a web-based digital credential verification system developed using the Django framework. The proposed system adopts a role-based architecture consisting of three primary entities: issuer, holder, and verifier, and employs RSA cryptographic techniques to ensure secure credential generation, authentication, and verification.

The scope of this work is limited to a prototype-level implementation designed for academic and experimental purposes. It does not include full-scale production deployment or complete blockchain integration. Instead, the system emphasizes the demonstration of distributed verification principles and the application of cryptographic security mechanisms within the context of digital identity management.

The proposed system is applicable in domains such as academic certificate verification, organizational identity validation, and controlled access environments where secure and reliable credential verification is required.

1.6 Significance

This study is significant as it contributes to the growing body of research in secure and distributed digital identity and credential verification systems. It examines how the integration of cryptographic techniques with distributed role-based architectures can enhance trust, data integrity, and security in digital credential verification processes.

The findings of this study are relevant to multiple stakeholders. For academic institutions and students, the proposed system provides a structured approach for secure issuance and verification of academic credentials. For organizations, it offers a framework that can help reduce credential fraud and prevent unauthorized use of identity information. For researchers, this study demonstrates a practical implementation of distributed verification principles using RSA cryptography and a web-based architecture.

Furthermore, this work provides a foundation for future research in advanced decentralized identity systems, including blockchain-based identity frameworks and zero-trust security models.

2 Literature Review

2.1 Centralized identity systems

Think about how you log into most websites. You type a username and password, and the website checks its database to see if you are who you say you are. Sometimes you use "Sign in with Google" or "Sign in with Facebook." In these cases, Google or Facebook acts as the identity provider. They store your credentials and tell the website whether you are allowed in.

This is called a centralized identity system. It works fine for many situations, but it has some well-known problems.

First, all your data is stored in one place. If that company gets hacked, and we see data breaches in the news all the time, your personal information can be stolen. For example, in 2019, Facebook exposed the phone numbers and personal data of over 400 million users. The data was found sitting on an open server with no password protection. Anyone who found the server could just download everything.

Second, you don't really control your own identity. The company can lock you out whenever they want. If Google decides to suspend your account, you lose access to every website where you used "Sign in with Google." A student at a university has a similar problem: the university issues your student ID and email, but if you graduate or get expelled, that identity disappears.

Thirdly, and this is the problem our project focuses on centralized systems often reveal too much information. When a website needs to check something simple like "is this user over 18?" the identity provider often sends back your full name, address, exact birth date, and other unnecessary details. A bouncer at a bar just needs to know you are old enough. They don't need your home address. But digital systems frequently ask for more data than necessary.

Our project tries to fix this third problem. We want to let users prove something about themselves like "I am over 18" or "I am a student" , without revealing everything else on their ID card.

2.2 Digital credentials

A credential is simply a statement made by one party about another party. Your driver's license is a credential issued by the government stating that you are allowed to drive. Your student ID is a credential issued by your university stating that you are enrolled.

Digital credentials are the electronic version of this. The World Wide Web Consortium (W3C) — the main organization that sets web standards — published a specification called Verifiable Credentials in 2019 (W3C, 2019). This specification defines how digital credentials should work in a way that is secure and privacy-respecting.

The W3C model has three roles:

- Issuer: The authority that creates and signs a credential. For a university, the registrar's office is the issuer. In our project, the Issuer Server Core running on port 8001 plays this role.
- Holder: The person who receives and stores the credential. This is usually a mobile wallet app. In our project, the Holder Device Wallet on port 8002 acts as the holder.

- **Verifier:** The person or system that checks the credential when it is presented. A website that needs to confirm your age is a verifier. In our project, the Verifier Audit Node on port 8003 plays this role.

The key idea is that these three roles are separate. The issuer does not need to talk to the verifier every time a credential is checked. The holder presents the credential directly to the verifier, and the verifier can check the cryptographic signature to confirm it really came from the issuer. This is much more efficient than the verifier calling the issuer for every single verification.

2.3 Distributed systems

Our project runs three separate Django services on three different ports: 8001 (issuer), 8002 (holder), and 8003 (verifier). These services communicate with each other over HTTP. This is a simple example of a distributed system — multiple computers (or in our case, multiple processes on the same computer) working together over a network.

Distributed systems have been studied for decades. One of the most famous ideas in this field is the CAP theorem, proposed by computer scientist Eric Brewer in 2000 and later proved by Seth Gilbert and Nancy Lynch (Gilbert & Lynch, 2002). The theorem says that a distributed system can only provide two out of these three guarantees:

- **Consistency** — every node sees the same data at the same time
- **Availability** — every request receives a response
- **Partition tolerance** — the system keeps working even if the network breaks

Network failures happen all the time. So any practical distributed system must be partition tolerant. This means you have to choose between consistency and availability.

Our project chooses availability. If the issuer server goes down, the holder wallet still works. The verifier might not be able to fetch the public key right away, but that is acceptable for an educational prototype. In a real production system, you would need to think more carefully about this trade-off.

2.4 Cryptography (RSA)

For a verifier to trust that a credential really came from the issuer, we need digital signatures. The issuer signs the credential with a private key, and anyone with the corresponding public key can verify the signature.

RSA is one of the oldest and most widely used digital signature algorithms. It was invented in 1977 by Ron Rivest, Adi Shamir, and Leonard Adleman at MIT (Rivest, Shamir, & Adleman, 1978). The security of RSA relies on a simple mathematical fact: multiplying two large prime numbers is easy, but taking the product and figuring out which two primes were used is extremely hard. This is called the factoring problem.

Our project also uses a cryptographic hash function called SHA256. A hash function takes any input and produces a fixed-length output that looks random. The important property for our project is that it is one-way — you cannot recover the input from the output (Preneel, 2010).

Our project uses a specific version of RSA called RSA-PSS. PSS stands for Probabilistic Signature Scheme. Unlike older methods, PSS adds a random value (called a salt) before signing. This means that even if you sign the exact same message twice, you get different signatures. This randomness makes certain types of attacks much harder.

The signing process in our project works like this:

1. For each claim (for example, "age: 27"), the issuer generates a random salt
2. The issuer computes $\text{SHA256}(\text{value} + \text{salt})$ — this produces a fixed-length hash
3. The issuer does this for every claim, then sorts all the hashes alphabetically
4. The issuer joins the sorted hashes with a special delimiter (a null byte)
5. The issuer signs this final string using RSA-PSS

When the holder wants to present the credential to a verifier, they can choose to hide some claims. For hidden claims, the holder only sends the hash — not the original value or the salt. The verifier then:

1. Recomputes the hash for any claim that was disclosed
2. Uses the provided hash for any claim that was hidden
3. Sorts the hashes the same way
4. Verifies the RSA-PSS signature

If the holder or anyone else changed any claim value or salt, the hash would change, and the signature verification would fail. This is how our system detects tampering.

2.5 Decentralized identity concepts

In recent years, researchers and practitioners have started talking about Self-Sovereign Identity (SSI). The basic idea is simple: people should own and control their own identities. You should not have to depend on Google, Facebook, or any government agency to prove who you are.

Christopher Allen, a well-known figure in the identity space, wrote an influential blog post in 2016 laying out the principles of SSI (Allen, 2016). He argued that identity systems must give users control over their personal data, allow them to share only what is necessary, and work across different services without being locked into a single provider.

Another important development in decentralized trust came from Bitcoin. In 2008, Satoshi Nakamoto published a paper describing a peer-to-peer electronic cash system that did not rely on any central authority (Nakamoto, 2008). While Bitcoin is about money, not identity, the idea of decentralizing trust inspired many later SSI systems.

Our project demonstrates one important SSI feature: selective disclosure. When the holder creates a presentation for the verifier, they can choose which claims to show and which to hide. For example, imagine a university student ID credential contains the following claims:

- Student Name: James Bond
- Student ID: 20250001
- Age: 27
- Enrollment Status: enrolled
- Department: Computer Science
- Home Address: 123 Main Street

If a professor only needs to verify that James is currently enrolled in the university, the holder can show only the Enrollment Status claim. The verifier never sees James's name, student ID, age, department, or home address. But the verifier can still check the issuer's signature to confirm that the Enrollment Status claim is authentic.

This is much better than traditional systems, where showing your physical student ID card reveals your name, photo, ID number, and validity date all at once — even if the person checking only needs to know whether you are a current student.

3 System Analysis & Design

3.1 Architecture diagram (issuer, holder, verifier)

3.1.1 Issuer

3.1.2 Holder

3.1.3 Verifier

3.2 Workflow explanation

3.3 Data flow diagrams

3.4 Security model

4 Implementation

4.1 Django framework

4.2 RSA implementation

4.3 Key storage

4.4 Modules

5. Discussion / Evaluation

6. Conclusion & Future Work

References

(National Institute of Standards and Technology (NIST), 2017)

(International Organization for Standardization (ISO), 2011)

(European Commission, 2020)

(Satybaldy, Hasselgren, & Nowostawski, 2022)

(World Wide Web Consortium (W3C), 2022)

Allen, C. (2016). The Path to Self-Sovereign Identity.

<http://www.lifewithalacrity.com/2016/04/the-path-to-self-sovereign-identity.html>

Gilbert, S., & Lynch, N. (2002). Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2), 51-59.

Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System.

<https://bitcoin.org/bitcoin.pdf>

Preneel, B. (2010). Cryptographic hash functions. In *Proceedings of the 15th European Symposium on Research in Computer Security* (pp. 3-4). Springer.

Rivest, R. L., Shamir, A., & Adleman, L. (1978). A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2), 120-126.